



Git in der Gruppe

Git in der Gruppe

Version 0.0.0 · Stand 2026-10-04 · Commit 7193b88+änderungen

© 2026 Jens · Lizenz: CC BY-SA 4.0



Inhalt

1 Git in der Gruppe	1
Anhang	11
A Glossar	12



1 Git in der Gruppe

Voraussetzungen: Git & GitHub, Markdown & README

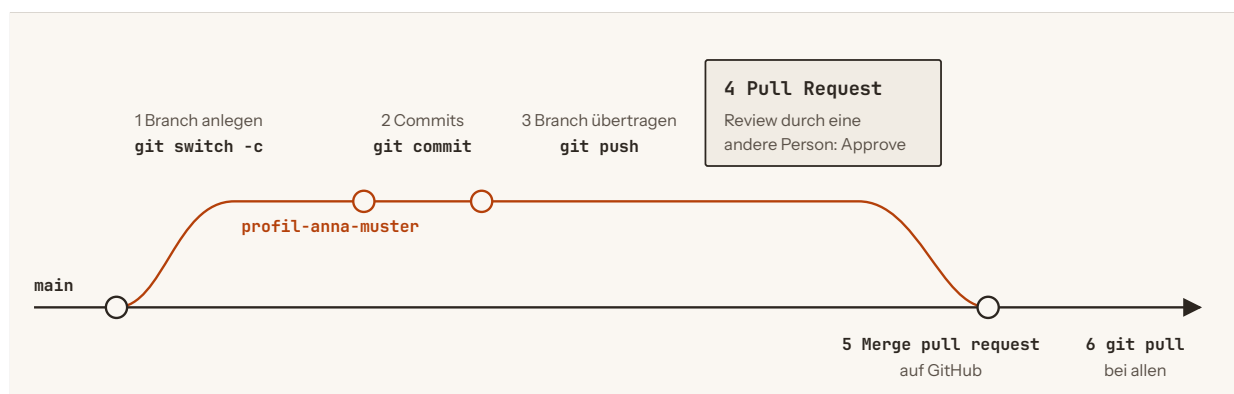
Lernziele

- Ein gemeinsames Repository einrichten und eine Änderung auf einem eigenen Branch über einen Pull Request nach main bringen, im Terminal und in VS Code.
- Einen Pull Request einer anderen Person prüfen und freigeben, den eigenen Pull Request nach der Freigabe mergen und main danach lokal aktualisieren.
- Einen Merge-Konflikt erkennen, in VS Code auflösen und den Pull Request danach mergen.

1. Vorschau

Zielstand nach diesem Kapitel: Die Gruppe arbeitet in einem gemeinsamen privaten Repository projekt auf GitHub. Jede Person hat darin mindestens drei Pull Requests gemergt, jeden nach der Freigabe durch eine andere Person. Die README hält die Regeln der Zusammenarbeit fest.

Das Modell hinter den Schritten dieses Kapitels, der GitHub Flow:



2. Warum

In einer Gruppe ändern mehrere Personen gleichzeitig dieselben Dateien. Ohne festen Ablauf überschreiben sie gegenseitig ihre Stände oder übernehmen fehlerhafte Änderungen ungeprüft. Der **GitHub Flow** legt jede Aufgabe auf einen eigenen Branch und übernimmt sie erst nach der Prüfung durch eine zweite Person nach `main`. Softwareteams arbeiten in Varianten dieses Ablaufs; ein Pull Request mit Review vor jedem Merge ist dort üblich.

Weiterführend: [GitHub Docs – GitHub flow](#) · [Pro Git, Kapitel 3: Git Branching](#)

3. Einrichten

Dieses Kapitel arbeitet in einem neuen Repository der Gruppe. Die Toolbox bleibt Einzelarbeit; sie bekommt am Ende nur die Notizen im Lernprotokoll. Wie in Git & GitHub nennt das Kapitel jeden Git-Schritt als Befehl im **Terminal** und als Aktion in **VS Code**. Schritte auf GitHub führst du im Browser aus.

Die Beispiele verwenden den Benutzernamen `anna-muster`. Setze an diesen Stellen deinen eigenen Benutzernamen ein.

3.1 Die Regeln der Gruppe

Die Gruppe arbeitet nach fünf Regeln:

Regel	Grund
1. Niemand committet direkt auf <code>main</code> .	<code>main</code> enthält nur geprüfte Stände. Jede Person kann jederzeit von <code>main</code> aus weiterarbeiten.
2. Jede Aufgabe bekommt einen eigenen Branch vom aktuellen <code>main</code> .	Aufgaben laufen parallel, ohne die Stände der anderen zu verändern.
3. Jeder Branch kommt über einen Pull Request nach <code>main</code> .	Der Pull Request zeigt alle Änderungen einer Aufgabe an einer Stelle.
4. Eine andere Person prüft den Pull Request und gibt ihn frei. Wer den Pull Request geöffnet hat, mergt ihn.	Eine zweite Person sieht jede Änderung. Den Zeitpunkt des Merge bestimmt die Person, die die Änderung kennt.
5. Wer den Pull Request gemergt hat, löscht den Branch. Alle aktualisieren <code>main</code> mit <code>git pull</code> .	Jeder neue Branch beginnt auf dem neuesten Stand.

GitHub setzt diese Regeln in der Einrichtung dieses Kapitels nicht durch. Jede Person der Gruppe kann direkt auf `main` pushen und ohne Freigabe mergen. Die Regeln sind eine Vereinbarung der Gruppe; [3.10](#) schreibt sie in die README.

3.2 Repository anlegen und einladen

Eine Person der Gruppe legt das Repository an. Sie ist **Owner** des Repositorys.

1. Wähle auf `github.com` **+** → **New repository**.
2. Setze den Namen auf `projekt` und die Sichtbarkeit auf **Private**.
3. Aktiviere die Option für eine README. Das Repository enthält dann einen ersten Commit mit der Datei `README.md`.
4. Wähle **Create repository**.
5. Wähle im Repository **Settings** → **Collaborators** (Bereich **Access**) → **Add people**.
6. Suche eine Person der Gruppe über ihren Benutzernamen und wähle **Add ... to projekt**. Wiederhole den Schritt für jede Person der Gruppe.

Die eingeladenen Personen erhalten eine E-Mail und nehmen die Einladung über den Link darin an. Danach können alle Personen der Gruppe in das Repository pushen.

3.3 Repository klonen

Jede Person legt eine Kopie des Repositorys auf ihrem Rechner an. Diesen Schritt nennt Git Clone. Der Klon enthält alle Commits und ist mit dem Repository auf GitHub als Remote origin verbunden.

Kopiere die Adresse auf der Seite des Repositorys über **Code** → **HTTPS**. Sie enthält den Benutzernamen des Owners.

Achtung: Lege den Klon neben dem Ordner toolbox ab, nicht darin. Liegt projekt in der Toolbox, meldet git status in der Toolbox den Ordner projekt/ unter Untracked files.

Terminal	VS Code
Wechsle mit <code>cd ..</code> aus der Toolbox in den Ordner darüber. Führe <code>git clone https://github.com/anna-muster/projekt.git</code> aus.	Command Palette → Git: Clone → Adresse einfügen → Enter → Ordner über toolbox wählen → Select as Repository Destination

Git antwortet mit `Cloning into 'projekt' ...` und legt den Ordner projekt an. Der Klon verwendet die Anmeldung bei GitHub aus Git & GitHub.

Öffne den Ordner projekt als Workspace. Im Terminal-Weg wählst du **File** → **Open Folder...**. VS Code fragt nach dem Klonen: „Would you like to open the repository?“ Wähle **Open**. Alle weiteren Schritte laufen in diesem Workspace.

3.4 Branch anlegen

Der Name eines Branch beschreibt die Aufgabe in kebab-case. Der erste Branch heißt profil- und dein Benutzername, zum Beispiel profil-anna-muster.

Terminal	VS Code
<code>git switch -c profil-anna-muster</code>	Branch-Name main in der Status Bar → Create new branch... → Name eingeben → Enter

Git antwortet mit `Switched to a new branch 'profil-anna-muster'`. Die **Status Bar** von VS Code zeigt den neuen Branch. Der Branch beginnt beim aktuellen Commit von main. Commits entstehen ab jetzt auf dem Branch; main bleibt unverändert.

3.5 Commit und Push des Branch

Lege im Workspace den Ordner team an und darin eine Datei mit deinem Benutzernamen, zum Beispiel anna-muster.md. Übernimm diesen Inhalt und setze deine Angaben ein:

```
# Anna Muster

- GitHub: anna-muster
- Aufgabe im Projekt: offen
- Erfahrung mit HTML und CSS: keine
- Erfahrung mit Git: Kapitel Git & GitHub
```

Jede Person der Gruppe ändert eine eigene Datei. Die vier Pull Requests dieses Abschnitts lassen sich deshalb in beliebiger Reihenfolge mergen.

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Changes) über der Liste Changes

Schritt	Terminal	VS Code
Prüfen	<code>git status</code> zeigt new file: team/anna-muster.md unter Changes to be committed	Die Datei steht unter Staged Changes
Commit	<code>git commit -m "docs: Profil von Anna Muster ergänzen"</code>	Nachricht eingeben → Commit
Übertragen	<code>git push -u origin profil-anna-muster</code>	Publish Branch

Der erste Push eines Branch legt ihn auf GitHub an. Git meldet * [new branch] profil-anna-muster → profil-anna-muster. Die Option -u verbindet den lokalen Branch mit dem Branch auf GitHub. Danach genügt `git push` bzw. **Sync Changes**.

3.6 Pull Request öffnen

Ein Pull Request ist eine Anfrage auf GitHub, die Commits eines Branch nach main zu übernehmen. Er zeigt alle Änderungen des Branch und sammelt Kommentare und Freigaben.

1. Öffne das Repository auf GitHub. Wähle im Hinweis über der Dateiliste **Compare & pull request**.
2. Prüfe die Zeile über dem Titel: **base** ist main, **compare** ist dein Branch.
3. Setze den Titel nach Conventional Commits, zum Beispiel docs: Profil von Anna Muster ergänzen.
4. Beschreibe im Feld darunter in einem Satz, was der Pull Request ändert.
5. Wähle rechts unter **Reviewers** eine Person der Gruppe.
6. Wähle **Create pull request**.

Fehlt der Hinweis, wähle **Pull requests** → **New pull request** und unter **compare** deinen Branch. GitHub nummeriert Pull Requests fortlaufend und listet sie im Tab **Pull requests**.

3.7 Review: prüfen und freigeben

Ein Review ist die Prüfung eines Pull Request durch eine andere Person. GitHub lässt die Freigabe des eigenen Pull Request nicht zu.

1. Öffne auf GitHub den Tab **Pull requests** und den Pull Request, für den du als Reviewer eingetragen bist.
2. Wähle den Tab **Files changed**. GitHub zeigt jede hinzugefügte und jede entfernte Zeile.
3. Prüfe den Pull Request gegen die Liste unten.
4. Wähle **Review changes** → **Approve** → **Submit review**.

Prüfe im Review:

- Der Pull Request enthält nur die Änderung, die sein Titel nennt.
- Die Commit-Nachrichten folgen Conventional Commits.
- Der Pull Request enthält keine Dateien des Betriebssystems wie .DS_Store (siehe Git & GitHub).

Erfüllt der Pull Request einen Punkt nicht, wähle statt **Approve** die Option **Request changes** und beschreibe die nötige Änderung im Kommentar. Die Person committet die Korrektur auf demselben Branch und überträgt sie mit `git push`. Der Pull Request zeigt den neuen Commit ohne weiteren Schritt.

3.8 Mergen

Öffne deinen Pull Request auf GitHub. Nach der Freigabe zeigt der Tab **Conversation** das Review mit dem Namen der prüfenden Person.

Achtung: Belasse die Methode im Menü neben der Schaltfläche **Merge pull request** auf **Create a merge commit**. Die anderen Methoden erzeugen keinen Merge-Commit.

1. Wähle unten im Tab **Conversation** die Schaltfläche **Merge pull request**.
2. Wähle **Confirm merge**.
3. Wähle **Delete branch**.

Die Schaltfläche **Merge pull request** erzeugt auf `main` einen Merge-Commit mit der Nachricht `Merge pull request #<Nummer> from <Owner>/<Branch>`. Die Nachricht erzeugt GitHub; sie folgt nicht Conventional Commits.

Delete branch löscht den Branch auf GitHub. Seine Commits bleiben über den Merge-Commit in `main` erhalten.

3.9 main aktualisieren

Den Merge-Commit enthält zunächst nur `main` auf GitHub. Nach jedem Merge, auch dem einer anderen Person, aktualisiert jede Person ihren lokalen `main`:

Schritt	Terminal	VS Code
Zu <code>main</code> wechseln	<code>git switch main</code>	Branch-Name in der Status Bar → <code>main</code>
Aktualisieren	<code>git pull</code>	Command Palette → Git: Pull
Eigenen Branch löschen	<code>git branch -d profil-anna-muster</code>	Command Palette → Git: Delete Branch... → Branch wählen

Pull holt die neuen Commits von GitHub und übernimmt sie in den aktuellen Branch. Ausgabe von `git pull` nach dem ersten Merge:

```
Updating 82f5be1..7f6dc8d
Fast-forward
 team/anna-muster.md | 6 +++++
 1 file changed, 6 insertions(+)
 create mode 100644 team/anna-muster.md
```

Fast-forward bedeutet: Der lokale `main` hatte keine eigenen Commits, Git setzt ihn auf den Stand von GitHub. Regel 1 sorgt dafür, dass `git pull` auf `main` immer so endet. `git branch -d` antwortet mit `Deleted branch profil-anna-muster (was ...)`.

Der nächste Branch beginnt mit 3.4 auf dem aktualisierten `main`.

3.10 Regeln in die README

Eine Person der Gruppe hält die Regeln aus 3.1 in der README fest. Sie folgt dem Ablauf von Branch anlegen bis main aktualisieren mit dem Branch `readme-zusammenarbeit` und der Commit-Nachricht `docs: Regeln der Zusammenarbeit ergänzen`. Ersetze den Inhalt von `README.md` durch:

projekt

Repository der Gruppe. Wer zur Gruppe gehört, steht im Ordner `team`.

Zusammenarbeit

1. Niemand committet direkt auf `main`.
2. Jede Aufgabe bekommt einen eigenen Branch vom aktuellen `main`.
3. Jeder Branch kommt über einen Pull Request nach `main`.
4. Eine andere Person prüft den Pull Request und gibt ihn frei. Wer den Pull Request geöffnet hat, mergt ihn.
5. Wer den Pull Request gemergt hat, löscht den Branch. Alle aktualisieren `main` mit `git pull`.

Commit-Nachrichten und Titel von Pull Requests folgen [Conventional Commits] (<https://www.conventionalcommits.org/en/v1.0.0/>).

3.11 Merge-Konflikt auflösen

Ändern zwei Branches dieselbe Stelle einer Datei, kann Git die Änderungen nicht automatisch zusammenführen. Das ist ein Merge-Konflikt. Der Pull Request meldet dann `This branch has conflicts that must be resolved` und bietet **Merge pull request** nicht an.

Den Konflikt löst die Person, die den Pull Request geöffnet hat, auf ihrem eigenen Branch:

Schritt	Terminal	VS Code
Zum eigenen Branch wechseln	<code>git switch <branch></code>	Branch-Name in der Status Bar → Branch wählen
Stand von GitHub abrufen	<code>git fetch</code>	Command Palette → Git: Fetch
main von GitHub übernehmen	<code>git merge origin/main</code>	Command Palette → Git: Merge... → origin/main

Fetch ruft die neuen Commits von GitHub ab und ändert keinen lokalen Branch. `git fetch` aktualisiert origin/main. `git merge origin/main` führt diesen Stand in deinen Branch. Bei einem Konflikt meldet Git:

```
Auto-merging team/anna-muster.md
CONFLICT (content): Merge conflict in team/anna-muster.md
Automatic merge failed; fix conflicts and then commit the result.
```

Git schreibt beide Fassungen der betroffenen Zeilen in die Datei, getrennt durch Konfliktmarker:

```
<<<<<< HEAD
Zeilen aus deinem Branch
=====
Zeilen aus origin/main
>>>>>> origin/main
```

So löst du den Konflikt auf:

1. Öffne die Datei. VS Code zeigt über jedem Konflikt die Aktionen **Accept Current Change** (dein Branch), **Accept Incoming Change** (origin/main) und **Accept Both Changes**.

2. Wähle eine Aktion oder bearbeite die Zeilen von Hand. Die Datei muss danach den gewünschten Inhalt haben und darf keinen Konfliktmarker mehr enthalten.
3. Speichere die Datei und übertrage die Auflösung:

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Merge Changes) über der Liste Merge Changes
Prüfen	<code>git status</code> zeigt <code>All conflicts fixed but you are still merging.</code>	Die Datei steht unter Staged Changes
Commit	<code>git commit -m "fix: Konflikt mit main auflösen"</code>	Nachricht eingeben → Commit
Übertragen	<code>git push</code>	Sync Changes

Der Pull Request meldet danach keinen Konflikt mehr. Das Review und der Merge folgen [3.7](#) und [3.8](#).

4. Verstehen

Öffne `learning-notizen.md` der Toolbox mit **File → Open File...** und lege die Überschrift **## Git in der Gruppe (Datum)** an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren. Die Experimente laufen im Repository `projekt` und ändern nur deine Datei im Ordner `team`. Ihre Änderungen bleiben in `main`.

Predict 1: Auf dem neuesten Stand

Ausgangsstand: Auf GitHub ist ein Pull Request gemergt, von dir oder einer anderen Person. Lokal hast du danach weder `git pull` noch `git fetch` ausgeführt.

- **Vorhersage:** Was meldet `git status` auf `main` in der zweiten Zeile?
- **Prüfen:**
 1. Wechsle zu `main` und führe `git status` aus.
 2. Führe `git fetch` aus und danach erneut `git status`.

→ [Auflösung 1](#) am Ende des Kapitels

Predict 2: Zwei Änderungen in einer Datei

Lege zwei Branches von `main` an, jeweils mit einem Commit in deiner Datei im Ordner `team`:

Branch	Änderung	Commit-Nachricht
<code>aufgabe-anna-muster</code>	Zeile - Aufgabe im Projekt: ... → - Aufgabe im Projekt: Startseite	<code>docs: Aufgabe von Anna Muster eintragen</code>
<code>erfahrung-anna-muster</code>	Zeile - Erfahrung mit HTML und CSS: ... → - Erfahrung mit HTML und CSS: ein Semester	<code>docs: Erfahrung von Anna Muster aktualisieren</code>

1. Lege den ersten Branch nach [3.4](#) an. Ändere die Zeile, erstelle den Commit und übertrage den Branch nach [3.5](#).
2. Wechsle zurück zu `main`. Lege den zweiten Branch an und verfare ebenso.
3. Öffne für beide Branches einen Pull Request mit einer Person der Gruppe als Reviewer.

- **Vorhersage:** Die beiden Branches ändern verschiedene Zeilen derselben Datei. Lässt sich der zweite Pull Request mergen, nachdem der erste gemergt ist?
- **Prüfen:** Merges den Pull Request zu aufgabe-anna-muster nach der Freigabe. Öffne danach den Pull Request zu erfahrung-anna-muster und betrachte den Bereich über der Schaltfläche **Merge pull request**.

→ Auflösung 2 am Ende des Kapitels

Break-it 1: Konfliktmarker im Commit

Führe für den Branch erfahrung-anna-muster die drei Schritte der ersten Tabelle aus 3.11 im Terminal aus. Ändere die Datei danach nicht.

- **Vorhersage:** Du überträgst die Datei mit den Konfliktmarkern unverändert in einen Commit. Lehnt Git den Commit ab, und was zeigt der Pull Request danach?
- **Prüfen:**

Achtung: Merges den Pull Request in diesem Zustand nicht.

1. Führe im Terminal `git add ., git commit -m "fix: Konflikt mit main auflösen"` und `git push` aus.
2. Öffne den Pull Request auf GitHub. Betrachte den Bereich über **Merge pull request** und den Tab **Files changed**.

→ Auflösung 3 am Ende des Kapitels

Commit

Öffne die Toolbox mit **File → Open Folder...**. Schließe die Notizen zu diesem Kapitel ab und übertrage sie. Formuliere die Nachricht selbst nach Conventional Commits.

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Changes) über der Liste Changes
Prüfen	<code>git status</code> zeigt die Dateien unter Changes to be committed	Die Dateien stehen unter Staged Changes
Commit	<code>git commit -m "<type>: <description>"</code>	Nachricht eingeben → Commit
Übertragen	<code>git push</code>	Sync Changes

5. Prüfen

- ☐ Der Tab **Pull requests** → **Closed** zeigt für jede Person der Gruppe mindestens drei gemergte Pull Requests, jeden mit der Freigabe einer anderen Person.
- ☐ Die README auf GitHub zeigt den Abschnitt „Zusammenarbeit“ mit fünf Regeln.
- ☐ `git log --oneline --graph` zeigt in projekt Merge-Commits mit der Nachricht Merge pull request #...
- ☐ `git branch` zeigt in projekt nur main.
- ☐ Nach `git fetch` meldet `git status` in projekt auf main die Zeilen Your branch is up to date with 'origin/main'. und nothing to commit, working tree clean.
- ☐ learning-notizen.md enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.

Zielstand des Repositorys projekt für eine Gruppe mit vier Personen:

```
projekt/  
├── .git/  
├── README.md  
└── team/  
    ├── anna-muster.md  
    ├── ben-beispiel.md  
    ├── carla-probe.md  
    └── david-test.md
```

Quellen

- [GitHub Docs – GitHub flow](#)
- [GitHub Docs – Inviting collaborators to a personal repository](#)
- [GitHub Docs – Creating a pull request](#)
- [GitHub Docs – Reviewing proposed changes in a pull request](#)
- [GitHub Docs – Merging a pull request](#)
- [VS Code – Merge conflicts](#)
- [VS Code – Branches and worktrees](#)
- [Pro Git \(Deutsch\), Kapitel 3: Git Branching](#) – Scott Chacon, Ben Straub

Auflösungen

Auflösung 1

Zu Predict 1: Auf dem neuesten Stand

Vor dem Abruf meldet `git status` die Zeile `Your branch is up to date with 'origin/main'`.

Nach `git fetch` lautet die Zeile `Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded`. Ein Pull Request mit einem Commit ergibt zwei neue Commits auf `main`: den Commit des Branch und den Merge-Commit.

`git status` fragt GitHub nicht ab. Es vergleicht `main` mit `origin/main`, dem Stand von GitHub beim letzten `git fetch` oder `git pull`. „Up to date“ bedeutet deshalb: gleich dem zuletzt abgerufenen Stand. Regel 5 verlangt deshalb `git pull`, bevor ein neuer Branch beginnt.

Führe `git pull` aus.

Auflösung 2

Zu Predict 2: Zwei Änderungen in einer Datei

Der zweite Pull Request meldet `This branch has conflicts that must be resolved`. Die beiden geänderten Zeilen liegen direkt untereinander. Git erkennt die Position einer Änderung an den unveränderten Zeilen davor und danach. Zwischen den beiden Änderungen liegt keine unveränderte Zeile, deshalb behandelt Git sie als eine Stelle. Liegt mindestens eine unveränderte Zeile dazwischen, führt Git beide Änderungen automatisch zusammen.

Lass den Pull Request offen. Break-it 1 setzt ihn fort.

Auflösung 3

Zu Break-it 1: Konfliktmarker im Commit

Git nimmt den Commit an. `git add` kennzeichnet die Datei als aufgelöst, ihren Inhalt prüft Git nicht. Die Konfliktmarker sind für Git gewöhnliche Textzeilen. Der Pull Request meldet keinen Konflikt mehr und bietet **Merge pull request** an. **Files changed** zeigt die Marker als neue Zeilen; ein Merge würde sie nach `main` übertragen. Beim Stagen einer Datei mit Konfliktmarkern zeigt VS Code eine Rückfrage mit dem Text „Are you sure you want to stage `anna-muster.md` with merge conflicts?“.

Korrigiere den Branch. Entferne die Marker und stelle beide Änderungen her: - Aufgabe im Projekt: Startseite und - Erfahrung mit HTML und CSS: ein Semester. Keine der drei Aktionen aus Merge-Konflikt auflösen ergibt diesen Inhalt. **Accept Both Changes** übernimmt beide Fassungen und verdoppelt die Zeilen. Übertrage die Korrektur mit der Commit-Nachricht `fix: Konfliktmarker entfernen` und `git push`. **Files changed** zeigt danach nur die geänderte Zeile zur Erfahrung. Merge den Pull Request nach der Freigabe und aktualisiere `main` nach 3.9. Lösche dabei beide Branches dieses Experiments.



Anhang



A Glossar

Begriffe aus dem Skript, alphabetisch sortiert. Englische Fachbegriffe bleiben englisch; die deutsche Übersetzung steht in der Definition. Die Kapitel verlinken auf diese Einträge.

Arbeitsweise

Lernprotokoll (`learning-notizen.md`)

Definition: Datei im Projekt der Toolbox. Sie enthält je Experiment die Vorhersage, das Ergebnis und die Erklärung. Sie ist der Nachweis des Verstehens und gehört nicht auf die Website.

Siehe auch: [Toolbox](#)

Lernziele (Kapitel)

Definition: Drei Stichpunkte am Anfang jedes Kapitels. Sie nennen, was du nach dem Kapitel kannst. Anhand der Lernziele entscheidest du, welche Teile du bearbeitest.

Pfad-Training

Definition: Übungsprojekt aus dem Material des Kapitels *Dateisystem & Pfade* mit drei Seiten und absichtlichen Pfadfehlern. Es verbindet die Kapitel *Dateisystem & Pfade*, *VS Code* und *Browser & DevTools*.

Siehe auch: [Pfad](#)

Toolbox

Definition: Das Lernprojekt der Kapitel zu HTML und CSS: eine Website, die Web-Patterns dokumentiert und mit genau diesen Patterns gebaut ist. Unterzeile „Bauteile für das Web“. Die Referenz-Toolbox zeigt die vollständige Toolbox.

Siehe auch: [Lernprotokoll](#)

Tooling

Definition: Das Handwerkszeug der Softwareentwicklung: Dateisystem, Editor, Browser-Dev-Tools und Git. Die Kapitel zu Tooling stehen vor den Kapiteln zu HTML und CSS.

Fachbegriffe

@starting-style

Definition: At-Rule mit Startwerten für eine Transition. Sie gilt, wenn ein Element zum ersten Mal dargestellt wird, zum Beispiel ein Popover beim Öffnen nach `display: none`. Ohne sie fehlt

der Wert vor der Änderung, und die Transition findet nicht statt.

Siehe auch: [Transition](#), [Popover](#), [MDN Web Docs](#)

@supports

Definition: At-Rule mit einer Bedingung. Die Regeln im Block gelten nur, wenn der Browser die Deklaration in der Bedingung kennt; mit not nur, wenn er sie nicht kennt. Eine Bedingung mit einer Custom Property trifft immer zu, weil eine Custom Property jeden Wert annimmt.

Siehe auch: [Progressive Enhancement](#), [Ersatzwert](#), [MDN Web Docs](#)

:has()

Definition: Pseudoklasse, die ein Element trifft, wenn ein Nachfahre dem Selektor in den Klammern entspricht. Beginnt der Selektor mit +, gilt er für das folgende Geschwister. `html:has(body > nav:popover-open)` trifft `<html>`, solange die Sidebar als Popover offen ist.

Siehe auch: [Selektor](#), [Popover](#), [MDN Web Docs](#)

Accessibility Tree

Definition: Vom Browser aus dem DOM abgeleitete Struktur, die assistiven Technologien (z. B. Screenreadern) Rolle, Name und Zustand jedes Elements bereitstellt. In den DevTools einsehbar.

Siehe auch: [DOM](#), [Landmark](#), [MDN Web Docs](#)

Akkordeon (exklusiv)

Definition: Mehrere Elemente `<details>` mit demselben Wert im Attribut `name`. Der Browser hält höchstens einen Eintrag der Gruppe offen, auch beim Laden der Seite. Die Zeile `<summary>` ist ein Tab-Stopp mit dem Zustand `expanded`.

Siehe auch: [Accessibility Tree](#), [MDN Web Docs](#)

Alt-Text

Definition: Der Text im Attribut `alt` eines `<img>`. Er ersetzt das Bild: Screenreader lesen ihn vor, und der Browser zeigt ihn an, wenn die Datei nicht lädt. Er ist der Name des Bildes im Accessibility Tree. Ein dekoratives Bild erhält `alt=""` und fehlt dann im Accessibility Tree.

Siehe auch: [Accessibility Tree](#), [Caption](#), [W3C Web Accessibility Initiative](#)

aria-current

Definition: Attribut, das in einer Gruppe verwandter Elemente das aktuelle kennzeichnet. Der Wert `page` kennzeichnet in einer Navigation den Link, der auf die geöffnete Seite verweist. Das Stylesheet der Toolbox leitet die sichtbare Markierung aus dem Attribut ab.

Siehe auch: [Main Navigation](#), [Breadcrumbs](#), [MDN Web Docs](#)

Anchor

Definition: Stelle einer Seite, auf die ein Link verweist: ein Element mit dem Attribut `id`. Der Link nennt `#` und die `id` (`href="#struktur"`), auf einer anderen Seite mit Pfad davor (`../index.html#struktur`).

Siehe auch: [Linktext](#), [Skip-Link](#), [MDN Web Docs](#)

Anfangswert

Definition: Wert, den CSS für eine Eigenschaft festlegt. Er gilt, wenn keine Deklaration die Eigenschaft für das Element festlegt und die Eigenschaft nicht vererbt wird, zum Beispiel

transparent für background-color oder none für outline.

Siehe auch: [Vererbung](#), [Custom Property](#), [MDN Web Docs](#)

Attribut

Definition: Angabe im öffnenden Tag eines Elements, geschrieben als Name und Wert: `lang="de", href="index.html"`. Ein Attribut ergänzt oder steuert das Element.

Siehe auch: [Element](#), [Tag](#), [MDN Web Docs](#)

Audit

Definition: Prüfung einer Seite gegen eine Liste von Erfolgskriterien. Das Audit-Protokoll (`audit.md`) nennt je Kriterium Stufe, Prüfung, Ergebnis mit Stelle und Maßnahme.

Siehe auch: [Erfolgskriterium](#), [WCAG](#)

Baseline

Definition: Einstufung eines Features der Web-Plattform nach seiner Verfügbarkeit in den verbreiteten Browsern Chrome, Edge, Firefox und Safari. Limited availability: Mindestens ein verbreiteter Browser unterstützt es nicht. Newly available: Die aktuelle Version jedes verbreiteten Browsers unterstützt es. Widely available: seit 30 Monaten Newly available. Der Name des Programms und die Namen der Stufen werden nicht übersetzt.

Siehe auch: [Progressive Enhancement](#), [Web Platform Status](#), [Can I use](#)

Box Model

Definition: Modell, nach dem der Browser jedes Element als Box darstellt: Inhalt, padding (Innenabstand), border (Rahmen) und margin (Außenabstand). box-sizing legt fest, ob width nur den Inhalt (content-box) oder die Box bis einschließlich border (border-box) meint.

Siehe auch: [Margin Collapsing](#), [MDN Web Docs](#)

Branch

Definition: Benannte Entwicklungslinie in einem Git-Repository. Ein neues Repository hat einen Branch, in diesem Skript `main`; Commits entstehen auf dem aktuellen Branch.

Siehe auch: [Commit](#), [Repository](#)

Breadcrumbs

Definition: Navigation, die die Lage einer Seite innerhalb der Website nennt: eine geordnete Liste (`<ol>`) von Links von der Startseite bis zur aktuellen Seite. Der englische Begriff wird nicht übersetzt. Der letzte Link trägt `aria-current="page"`.

Siehe auch: [aria-current](#), [Pattern](#), [W3C Web Accessibility Initiative](#)

Breakpoint

Definition: Breite des Viewports, ab der die Bedingung einer Media Query zutrifft. Die Toolbox hat einen Breakpoint bei 64rem: Ab dieser Breite steht die Sidebar als Spalte neben dem Hauptinhalt.

Siehe auch: [Media Query](#), [Mobile-first](#)

Caption

Definition: Sichtbare Beschriftung eines Bildes im Element `<figcaption>` innerhalb von `<figure>` oder einer Tabelle im Element `<caption>`. Die Caption eines Bildes ergänzt es für alle;

der Alt-Text ersetzt es. Die Caption einer Tabelle ist ihr Name im Accessibility Tree.

Siehe auch: [Alt-Text](#), [Header-Zelle](#), [MDN Web Docs](#)

Cascade Layers

Definition: Ebenen eines Stylesheets, festgelegt mit `@layer`. Eine Regel in einer späteren Ebene gewinnt gegen eine Regel in einer früheren Ebene, unabhängig von der Spezifität; eine Regel außerhalb jeder Ebene gewinnt gegen alle Ebenen. Im Fließtext heißt die einzelne Schicht „Ebene“. Die Toolbox hat die Ebenen `tokens`, `base`, `layout`, `components`, `overrides`.

Siehe auch: [Kaskade](#), [Spezifität](#), [MDN Web Docs](#)

clamp()

Definition: CSS-Funktion mit drei Angaben: Minimum, Wert und Maximum. Der Wert wächst zum Beispiel mit der Breite des Viewports (`vw`) und bleibt zwischen den zwei Grenzen. Eine Größe ändert sich so ohne Breakpoint.

Siehe auch: [Breakpoint](#), [Custom Property](#)

Clone

Definition: Kopie eines Repositorys von einem Remote auf den eigenen Rechner mit `git clone`, in VS Code **Git: Clone**. Der Klon enthält alle Commits und ist mit dem Remote als `origin` verbunden.

Siehe auch: [Remote](#), [Repository](#)

Code-Block

Definition: Block in Markdown zwischen zwei Zeilen mit mindestens drei Backticks; Zeilenumbrüche und Leerzeichen bleiben erhalten. Nach den öffnenden Backticks steht optional die Sprache. In HTML entspricht er `<pre><code>`.

Siehe auch: [Markdown](#)

Color Scheme

Definition: Helles oder dunkles Farbschema einer Seite: Light Mode oder Dark Mode. Die Eigenschaft `color-scheme` nennt die Color Schemes, die ein Element unterstützt; die Media Query `prefers-color-scheme` fragt ab, welches die Person im Betriebssystem eingestellt hat. Der englische Begriff wird nicht übersetzt.

Siehe auch: [light-dark\(\)](#), [Media Query](#), [MDN Web Docs](#)

Commit

Definition: Gespeicherter Stand eines Git-Repositorys mit Nachricht, Autor:in und eindeutiger Kennung (Hash). Entsteht in zwei Schritten: `git add` (in die Staging Area), `git commit`.

Siehe auch: [Repository](#), [Staging Area](#)

Container Query

Definition: Bedingung im Stylesheet (`@container`), unter der Regeln gelten, zum Beispiel ab einer Breite des Query Containers. Eine Komponente passt sich damit an den Platz an ihrem Ort an, nicht an den Viewport.

Siehe auch: [Query Container](#), [Media Query](#)

Container und Item

Definition: Ein Container ist ein Element mit `display: flex` (Flex-Container) oder `display: grid` (Grid-Container). Seine Kinder sind die Items. Eigenschaften des Containers legen die Anordnung aller Items fest; Enkel sind keine Items.

Siehe auch: [Flexbox](#), [Grid](#)

Conventional Commits

Definition: Konvention für Commit-Nachrichten der Form `<type>: <description>`, z. B. `fix: Pfad zum Logo korrigieren`. Die Spezifikation definiert `feat` und `fix` und lässt weitere Typen zu.

Siehe auch: [Commit](#), [Conventional Commits 1.0.0](#)

CSS

Definition: Cascading Style Sheets – regelbasierte Sprache für die Darstellung von HTML. Ein Stylesheet besteht aus Regeln; der Browser wendet jede Regel auf alle Elemente an, auf die ihr Selektor zutrifft.

Siehe auch: [Regel](#), [Stylesheet](#), [Kaskade](#), [MDN Web Docs](#)

Custom Property

Definition: Eigenschaft mit frei gewähltem Namen, der mit `--` beginnt (`--accent-hue: 45;`). Die Funktion `var()` setzt ihren Wert in eine Deklaration ein. Custom Properties werden vererbt; der Browser prüft den Wert erst beim Einsetzen.

Siehe auch: [Deklaration](#), [MDN Web Docs](#)

Dateiendung

Definition: Teil des Dateinamens nach dem letzten Punkt (`.html`, `.css`, `.md`). Bestimmt, wie Programme die Datei behandeln. Windows und macOS blenden sie standardmäßig aus – für Webprojekte immer einblenden.

Deklaration

Definition: Paar aus Eigenschaft und Wert in einer Regel eines Stylesheets, z. B. `font-size: var(--text-xs);`. Eine Regel besteht aus einem Selektor und einem Block von Deklarationen.

Siehe auch: [Selektor](#), [Kaskade](#)

DevTools

Definition: In den Browser eingebaute Entwicklerwerkzeuge (Elements, Console, Network, Accessibility ...), die zeigen, was der Browser geladen und aus dem Code gemacht hat. Änderungen dort wirken nur im DOM, nicht in der Datei.

Siehe auch: [DOM](#)

DOM

Definition: Document Object Model – die Baumstruktur, die der Browser aus dem HTML-Quelltext erzeugt. Kann vom Quelltext abweichen, weil der Browser Fehler stillschweigend korrigiert.

Siehe auch: [Accessibility Tree](#)

Element

Definition: Baustein eines HTML-Dokuments aus öffnendem Tag, Inhalt und schließendem Tag, zum Beispiel `<title>Toolbox</title>`. Leere Elemente wie `<meta>` und `<link>` haben keinen In-

halt und keinen schließenden Tag.

Siehe auch: [Tag](#), [Attribut](#), [DOM](#), [MDN Web Docs](#)

Erfolgskriterium

Definition: Prüfbare Aussage der WCAG mit Nummer und Stufe (A, AA, AAA), zum Beispiel 1.4.3 Contrast (Minimum), Stufe AA. Die Nummer nennt Prinzip, Richtlinie und Kriterium. Englisch: Success Criterion.

Siehe auch: [WCAG](#), [Audit](#), [W3C Web Accessibility Initiative](#)

Ersatzwert

Definition: Wert für den Fall, dass ein anderer Wert nicht gilt. Als zweite Angabe in `var()` gilt er, wenn die Custom Property nicht definiert ist. Als Deklaration vor einer Deklaration mit einem neueren Feature gilt er in Browsern ohne das Feature. Vor einer Deklaration mit `var()` wirkt er nicht: Der Browser prüft den eingesetzten Wert erst nach der Kaskade.

Siehe auch: [Custom Property](#), [@supports](#), [Kaskade](#)

Fetch

Definition: Abruf neuer Commits von einem Remote mit `git fetch`. Fetch aktualisiert `origin/main` und ändert keinen lokalen Branch.

Siehe auch: [Pull](#), [Remote](#)

Flexbox

Definition: Layout-Verfahren für Items in einer Richtung (Hauptachse): in einer Zeile oder in einer Spalte, mit Umbruch (`flex-wrap`) und Verteilung des freien Platzes je Zeile (`justify-content`). Der Inhalt der Items bestimmt die Aufteilung. Die Querachse verläuft senkrecht zur Hauptachse.

Siehe auch: [Container und Item](#), [Grid](#), [Gap](#), [MDN Web Docs](#)

Fokus

Definition: Das Element, an das Eingaben über die Tastatur gehen. Ein Klick oder die Taste Tab setzt ihn; der Browser zeigt ihn meist als Rahmen um das Element.

Siehe auch: [Tab-Stopp](#)

Gap

Definition: Abstand zwischen den Items eines Flex- oder Grid-Containers, festgelegt mit `gap`. Das Gap steht nur zwischen Items, nicht am Rand des Containers.

Siehe auch: [Flexbox](#), [Grid](#)

Git

Definition: Werkzeug zur Versionskontrolle: speichert Stände (Commits) eines Projekts lokal im Ordner `.git`. GitHub, GitLab oder Codeberg sind Plattformen, auf denen Repositories online liegen.

Siehe auch: [Commit](#), [Remote](#)

Grid

Definition: Layout-Verfahren mit Spalten und Zeilen, die der Container für alle Items festlegt (`grid-template-columns`, `grid-template-rows`). Die Einheit `fr` ist ein Anteil am freien Platz;

`minmax()` nennt die kleinste und die größte Größe einer Spalte oder Zeile.

Siehe auch: [Grid Area](#), [Flexbox](#), [Container](#) und [Item](#), [MDN Web Docs](#)

Grid Area

Definition: Bereich eines Grids mit einem Namen. `grid-template-areas` legt die Namen im Container fest, `grid-area` ordnet ein Item einem Grid Area zu. Das Seitenlayout der Toolbox hat die Grid Areas `header`, `main` und `footer`.

Siehe auch: [Grid](#), [Landmark](#)

Header-Zelle

Definition: Zelle einer Tabelle, die eine Spalte oder eine Zeile benennt: das Element `<th>`. Das Attribut `scope` nennt die Richtung (`col`, `row`). Im Accessibility Tree hat sie die Rolle `columnheader` oder `rowheader`; eine Datenzelle (`<td>`) hat die Rolle `cell`.

Siehe auch: [Caption](#), [Accessibility Tree](#)

Kaskade

Definition: Verfahren, mit dem der Browser aus mehreren Deklarationen derselben Eigenschaft für ein Element eine auswählt. In den Materialien in drei Schritten: Ebene, Spezifität, Reihenfolge im Quelltext.

Siehe auch: [Cascade Layers](#), [Spezifität](#), [MDN Web Docs](#)

Klasse

Definition: Selbst gewählter Name im Attribut `class` eines Elements (`class="code"`). Der Selektor `.code` wählt jedes Element mit dieser Klasse aus. Der Name nennt die Aufgabe, nicht das Aussehen.

Siehe auch: [Selektor](#), [Komponente](#), [Spezifität](#), [MDN Web Docs](#)

Komponente

Definition: Wiederverwendbarer Baustein einer Seite mit eigenem HTML-Aufbau und eigenen Regeln im Stylesheet, ausgewählt über eine Klasse (`.code`). Der Name der Klasse nennt die Aufgabe, nicht das Aussehen.

Siehe auch: [Pattern](#), [Klasse](#)

Kontrast

Definition: Verhältnis der Helligkeit zweier Farben zwischen 1:1 und 21:1, berechnet nach WCAG. Text braucht zum Hintergrund mindestens 4,5:1, großer Text (ab 24 px oder ab 18,66 px fett) 3:1. Die DevTools nennen den Wert im Tooltip des **Inspect mode**.

Siehe auch: [oklch\(\)](#), [W3C Web Accessibility Initiative](#)

Label

Definition: Das Element `<label>`: der sichtbare Text zu einem Feld eines Formulars. Über `for` und `id` ist er der Name des Felds im Accessibility Tree; ein Klick auf das Label setzt den Fokus in das Feld. Das Attribut `name` ist dagegen der Schlüssel des Werts in den gesendeten Daten.

Siehe auch: [Placeholder](#), [Accessibility Tree](#)

Landmark

Definition: Seitenbereich mit fester Rolle im Accessibility Tree, zu dem Screenreader direkt springen können. HTML-Elemente → Rollen: `<header>` → `banner`, `<nav>` → `navigation`, `<main>` →

main, <footer> → contentinfo (header/footer nur als direkte Kinder von <body>).

Siehe auch: [Accessibility Tree](#)

Layout Shift

Definition: Verschiebung von sichtbarem Inhalt, nachdem die Seite dargestellt ist, zum Beispiel weil ein Bild ohne reservierte Fläche lädt. Die Attribute width und height am reservieren die Fläche vor dem Laden.

Siehe auch: [Lazy Loading](#), [MDN Web Docs](#)

Lazy Loading

Definition: Aufgeschobenes Laden: Mit loading="lazy" fordert der Browser eine Bilddatei erst an, wenn das Bild in die Nähe des Viewports kommt. Wirkt nur auf Seiten, die mehrere Viewports lang sind.

Siehe auch: [Layout Shift](#), [Viewport](#), [web.dev](#)

light-dark()

Definition: CSS-Funktion mit zwei Farben: Sie liefert im Light Mode die erste und im Dark Mode die zweite. Sie wirkt nur mit color-scheme und wird an dem Element ausgewertet, dessen Deklaration die Farbe verwendet.

Siehe auch: [Color Scheme](#), [Token](#), [MDN Web Docs](#)

Linktext

Definition: Der Text zwischen <a> und . Er ist der Name des Links im Accessibility Tree und nennt das Ziel so, dass er ohne den umgebenden Satz verständlich ist.

Siehe auch: [Accessibility Tree](#)

Main Navigation

Definition: Die Navigation im Header jeder Seite mit den Zielen der Website. Enthält eine Seite mehrere Elemente <nav>, erhält jedes über aria-label einen Namen; in der Toolbox der Studierenden trägt sie den Namen „Main“.

Siehe auch: [Landmark](#), [Breadcrumbs](#)

Margin Collapsing

Definition: Zusammenfallen der Margins zweier Block-Boxen untereinander: Der Abstand ist der größere der beiden Werte, nicht ihre Summe. Padding und Rahmen fallen nie zusammen.

Siehe auch: [Box Model](#), [MDN Web Docs](#)

Markdown

Definition: Textformat, das die Struktur eines Textes (Überschriften, Listen, Links, Code, Tabellen) mit Zeichen im Text kennzeichnet. Das Skript folgt CommonMark, Tabellen stammen aus GitHub Flavored Markdown (GFM).

Siehe auch: [README](#), [Code-Block](#), [CommonMark Spec 0.31.2](#)

Media Query

Definition: Bedingung im Stylesheet (@media), unter der Regeln gelten, zum Beispiel ab einer Breite des Viewports. Ein Grid mit repeat(auto-fill, minmax()) passt die Spaltenzahl ohne Media Query an den Platz an.

Siehe auch: [Breakpoint](#), [Mobile-first](#), [Container Query](#), [Grid](#), [Viewport](#)

Merge

Definition: Zusammenführen der Commits eines Branch in einen anderen Branch. Ein Merge-Commit hat zwei Vorgänger: den letzten Commit beider Branches.

Siehe auch: [Branch](#), [Merge-Konflikt](#), [Pull Request](#)

Merge-Konflikt

Definition: Zustand, in dem Git zwei Änderungen an derselben Stelle einer Datei nicht automatisch zusammenführen kann. Git schreibt beide Fassungen mit Konfliktmarkern in die Datei; eine Person entscheidet über den Inhalt.

Siehe auch: [Merge](#)

Mobile-first

Definition: Reihenfolge im Stylesheet: Die Regeln ohne Bedingung beschreiben die schmale Form der Seite. Jede Media Query ergänzt Regeln ab einem Breakpoint und steht nach ihnen.

Siehe auch: [Media Query](#), [Breakpoint](#)

oklch()

Definition: CSS-Funktion für eine Farbe aus Helligkeit (0 % bis 100 %), Chroma (0 für Grau bis etwa 0,4) und Farbton (Winkel von 0 bis 360). Die Helligkeit folgt der Wahrnehmung: Farben mit gleicher Helligkeit haben zu einem Hintergrund einen ähnlichen Kontrast.

Siehe auch: [Kontrast](#), [MDN Web Docs](#)

Override-Datei (overrides.css)

Definition: Datei `assets/css/overrides.css` der Toolbox für eigene Gestaltung. Sie gehört zur Ebene `overrides`, die in `main.css` zuletzt steht; eine Regel darin gewinnt gegen die Regeln aller anderen Dateien. Eigene Gestaltung ist freiwillig. Die Kapitel ändern diese Datei nicht.

Siehe auch: [Cascade Layers](#), [Custom Property](#)

Pattern

Definition: Bewährte, wiederverwendbare Lösung für ein wiederkehrendes Problem – hier: ein Baustein einer Webseite (z. B. Breadcrumbs, Landmarks) mit festem HTML-Aufbau und bekannten Regeln zur Barrierefreiheit.

Siehe auch: [Toolbox](#)

Pfad

Definition: Weg zu einer Datei. **Relativ** (`../assets/img/logo.svg`) von der Datei aus, in der er steht – funktioniert, wo auch immer das Projekt liegt. **Absolut** (`C:\Users\.../Users/...`) vom obersten Ordner aus (Windows: Laufwerk, macOS: `/`) – funktioniert nur auf einem Rechner. In HTML und CSS für Dateien des eigenen Projekts immer relativ und immer mit `/`. Ein Link auf eine andere Website enthält eine absolute URL mit Schema (`https://git-scm.com/`); sie ist kein Pfad auf dem eigenen Rechner.

Pflichtfeld

Definition: Feld eines Formulars mit dem Attribut `required`. Der Browser sendet das Formular nur ab, wenn das Feld einen Wert enthält, und zeigt sonst eine Fehlermeldung.

Siehe auch: [Validierung](#)

Placeholder

Definition: Das Attribut `placeholder`: ein Beispieltext in einem leeren Feld. Er verschwindet mit der ersten Eingabe und ersetzt kein Label.

Siehe auch: [Label](#)

Popover

Definition: Element mit dem Attribut `popover`. Der Browser stellt es nur dar, solange es offen ist; offen liegt es über allen anderen Elementen der Seite. Ein Button mit `popover-target` öffnet und schließt es, Escape und ein Klick daneben schließen es. Die Pseudoklasse `:popover-open` trifft ein offenes Popover.

Siehe auch: [:has\(\)](#), [@starting-style](#), [MDN Web Docs](#)

Progressive Enhancement

Definition: Vorgehen, bei dem eine Seite mit den Features funktioniert, die jeder Browser unterstützt, und neuere Features sie in Browsern mit Unterstützung verbessern. Ein Feature ohne Unterstützung fällt ohne Schaden weg oder erhält einen Ersatz mit `@supports`. Der englische Begriff wird nicht übersetzt.

Siehe auch: [Baseline](#), [@supports](#), [MDN Web Docs](#)

Pull

Definition: Abruf neuer Commits von einem Remote und Übernahme in den aktuellen Branch mit `git pull`, in VS Code **Git: Pull**. Pull besteht aus Fetch und Merge.

Siehe auch: [Fetch](#), [Push](#)

Pull Request

Definition: Anfrage auf einer Plattform wie GitHub, die Commits eines Branch in einen anderen Branch zu übernehmen, meist nach `main`. Zeigt alle Änderungen und sammelt Kommentare, Reviews und Freigaben.

Siehe auch: [Review](#), [Merge](#), [Branch](#)

Push

Definition: Übertragen der lokalen Commits zu einem Remote mit `git push`, in VS Code **Sync Changes**. Danach zeigt etwa GitHub denselben Stand.

Siehe auch: [Remote](#), [Commit](#)

Query Container

Definition: Element mit der Deklaration `container-type`, dessen Breite eine Container Query abfragt. Der Browser berechnet seine Breite ohne den Inhalt. Der Begriff unterscheidet das Element vom Flex-Container und vom Grid-Container.

Siehe auch: [Container Query](#), [Container und Item](#)

README

Definition: Datei `README.md` im obersten Ordner eines Projekts. Sie beschreibt Zweck, Aufbau und Start des Projekts für Personen, die es nicht angelegt haben; GitHub zeigt sie auf der Startseite des Repositorys.

Siehe auch: [Markdown](#), [Repository](#)

Regel

Definition: Baustein eines Stylesheets aus einem Selektor und einem Block von Deklarationen in geschweiften Klammern: `figcaption { color: var(--color-muted); }`.

Siehe auch: [Selektor](#), [Deklaration](#), [CSS](#)

Remote

Definition: Verbundene Kopie eines Repositorys auf einem anderen Rechner, meist einer Plattform wie GitHub. Standardname `origin`; `git push` überträgt Commits dorthin.

Siehe auch: [Repository](#)

Repository

Definition: Ein von Git verwaltetes Projekt: Arbeitsordner plus die gesamte Versionsgeschichte im Ordner `.git`. In das Repository gelangt, was ein Commit enthält; die Commits liegen im Ordner `.git`.

Siehe auch: [Commit](#), [Git](#)

Responsive Images

Definition: Mehrere Dateien für ein Bild, aus denen der Browser eine wählt. In der Toolbox: `<picture>` mit `<source media>` liefert je nach Breite des Viewports eine andere Datei; dargestellt wird immer das `<img>`.

Siehe auch: [Viewport](#), [Alt-Text](#), [MDN Web Docs](#)

Review

Definition: Prüfung eines Pull Request durch eine andere Person vor dem Merge. Ergebnis ist ein Kommentar, eine Freigabe (**Approve**) oder die Bitte um Änderungen (**Request changes**).

Siehe auch: [Pull Request](#)

Selektor

Definition: Teil einer Regel, der die Elemente auswählt, für die die Regel gilt: ein Element (`a`), eine Klasse (`.code`), ein Attribut (`[aria-current="page"]`), eine Pseudoklasse (`:focus-visible`) oder eine Kombination (`nav a: jedes <a> in <nav>; main > p: jedes <p> direkt in <main>; * + *: jedes Element direkt nach einem anderen`). Eine Liste mit Komma (`main > * + *, article > * + *`) gilt für jeden ihrer Selektoren.

Siehe auch: [Spezifität](#), [Deklaration](#)

Sidebar

Definition: Spalte neben dem Hauptinhalt. In der Toolbox enthält sie die Navigation mit allen Patterns (`<nav aria-label="Patterns">`). Ab 64rem steht sie als Spalte neben dem Hauptinhalt; darunter öffnet der Button „Patterns“ sie als [Popover](#).

Siehe auch: [Main Navigation](#), [Breakpoint](#)

Skip-Link

Definition: Link am Anfang jeder Seite, der zum Hauptinhalt führt (`href="#inhalt"`). Er ist der erste [Tab-Stopp](#) und erscheint erst mit dem Fokus.

Siehe auch: [Landmark](#), [W3C Web Accessibility Initiative](#)

Spezifität

Definition: Gewicht eines Selektors aus drei Zahlen: IDs, dann Klassen, Attribute und Pseudo-klassen, dann Elemente und Pseudoelemente (nav ol a = 0,0,3; .code = 0,1,0). Sie entscheidet zwischen zwei Regeln derselben Ebene.

Siehe auch: [Kaskade](#), [Selektor](#), [MDN Web Docs](#)

Staging Area

Definition: Zwischenbereich in Git für Änderungen, die in den nächsten Commit sollen (git add). Erlaubt, zusammengehörige Änderungen gezielt zu einem Commit zu bündeln.

Siehe auch: [Commit](#)

Stylesheet

Definition: Datei mit CSS-Regeln, meist mit der Endung .css. Eine HTML-Seite lädt sie mit `<link rel="stylesheet" href="...">`; findet der Browser die Datei nicht, erscheint die Seite ohne Gestaltung. Regeln für nur eine Seite stehen im Element `<style>`, Deklarationen für nur ein Element im Attribut `style`.

Siehe auch: [Pfad](#), [Selektor](#), [CSS](#)

Tab-Stopp

Definition: Element, das der Fokus mit Tab erreicht: Links, Buttons, Felder und Elemente mit `tabindex="0"`. Eine Gruppe von Radio Buttons ist ein Tab-Stopp; die Pfeiltasten wählen darin.

Siehe auch: [Skip-Link](#), [W3C Web Accessibility Initiative](#)

Tag

Definition: Auszeichnung in spitzen Klammern, die ein Element beginnt (`<title>`) oder beendet (`</title>`). Element und Tag sind nicht dasselbe: Das Element umfasst beide Tags und den Inhalt dazwischen.

Siehe auch: [Element](#)

Token

Definition: Custom Property für einen Wert der Gestaltung: Farbe, Schrift, Schriftgröße, Zeilenhöhe, Abstand oder Maß. Der Name nennt die Aufgabe, nicht das Aussehen (`--color-paper`, nicht `--color-white`). Die Toolbox definiert alle Tokens in `tokens.css`; das Pattern heißt „Design-Tokens“.

Siehe auch: [Custom Property](#), [Design Tokens Community Group](#)

Transition

Definition: Übergang einer Eigenschaft von einem Wert zu einem anderen über eine Dauer, festgelegt mit `transition`. Eigenschaften ohne Zwischenwerte wie `display` wechseln nur mit `allow-discrete` erst am Ende der Dauer. Personen, die im System Bewegung reduziert haben, erhalten in der Toolbox keine Transition (`prefers-reduced-motion`).

Siehe auch: [@starting-style](#), [MDN Web Docs](#)

Überschriften-Hierarchie

Definition: Gliederung einer Seite durch `<h1>` bis `<h6>`: genau ein `<h1>`, keine übersprungene Ebene, Ebene nach Struktur statt nach Schriftgröße.

Siehe auch: [Accessibility Tree](#)

Validierung

Definition: Prüfung der Eingaben eines Formulars durch den Browser vor dem Absenden, gesteuert über Attribute wie `required`, `type` und `pattern`. Der Selektor `:invalid` trifft auf ein Feld mit ungültigem Wert zu, `:user-invalid` erst nach einer Eingabe der Person oder nach dem Absenden.

Siehe auch: [Pflichtfeld](#)

Vererbung

Definition: Ein Element übernimmt den Wert einer Eigenschaft vom umgebenden Element, solange keine Regel die Eigenschaft für das Element selbst festlegt. Custom Properties, Textfarbe und Schrift werden vererbt, Abstände und Hintergrund nicht. Eine Custom Property wird mit ihrem berechneten Wert vererbt: `var()` ist darin eingesetzt, eine Funktion wie `light-dark()` bleibt stehen.

Siehe auch: [Custom Property](#), [Kaskade](#), [MDN Web Docs](#)

Viewport

Definition: Der sichtbare Bereich der Seite im Browserfenster. Ohne `<meta name="viewport">` rechnen Mobilbrowser mit einer virtuellen Breite von ca. 980 px und verkleinern die Seite.

WCAG

Definition: Web Content Accessibility Guidelines des W3C, aktuell in der Fassung 2.2: vier Prinzipien, 13 Richtlinien und Erfolgskriterien mit den Stufen A, AA und AAA. BFGS und BITV 2.0 verlangen über EN 301 549 die Stufe AA.

Siehe auch: [Erfolgskriterium](#), [Kontrast](#), [W3C Web Accessibility Initiative](#)

Wordmark

Definition: Name einer Website als gestalteter Schriftzug, meist oben links im Header und als Link auf die Startseite. In der Toolbox ist es der Link „Toolbox“ mit dem farbigen Quadrat.

Siehe auch: [Landmark](#)

Workspace

Definition: Ein in VS Code geöffneter Ordner. Explorer, Pfad-Vervollständigung und Terminal beziehen sich auf diesen Ordner.

Zeichenkodierung

Definition: Regel, nach der ein Programm die gespeicherten Bytes einer Datei in Zeichen übersetzt. UTF-8 enthält alle Zeichen, auch Umlaute wie „ä“ und „ß“. Eine HTML-Seite nennt sie mit `<meta charset="utf-8">`.

Siehe auch: [MDN Web Docs](#)